

TDE – SISTEMAS DISTRIBUÍDOS

Sistema Distribuído de Consulta de Filas do SAC Bahia

Opção 2 – REST (OpenAPI/Swagger)

Disciplina	Sistemas Distribuídos
Projeto	Fila de Espera do SAC Bahia
Arquitetura	Cliente Python + API Node.js/Express + SQLite
Comunicação	HTTP/REST com dados em JSON
Documentação da API	OpenAPI/Swagger

Planejamento inicial do projeto

Integrantes da equipe:

João Pedro Vinhas

Estevão Cunha

Guilherme Fernandes

Matheus da Silva

Ulysses Silva

1. Ideia do Projeto

O projeto consiste no desenvolvimento de um sistema distribuído para consulta do tempo de espera em diferentes postos do SAC Bahia. O usuário utilizará um cliente desenvolvido em Python, executado no terminal, para informar o ID de um posto. O cliente enviará uma requisição HTTP para uma API REST desenvolvida em Node.js com Express. A API consultará os dados persistidos em SQLite e retornará, em formato JSON, informações do posto e da fila, como quantidade de pessoas aguardando e tempo estimado de espera.

2. Problema e Justificativa

O tempo de espera é uma informação importante para usuários de serviços presenciais. A proposta simula um ambiente no qual diferentes aplicações, implementadas em linguagens distintas, precisam trocar dados pela rede de forma padronizada. O cenário permite aplicar conceitos de Sistemas Distribuídos sem introduzir complexidade desnecessária, concentrando o trabalho na comunicação entre serviços e no uso correto de REST.

3. Objetivo Geral

Implementar uma solução distribuída em que um cliente Python consulte uma API REST em Node.js para obter informações sobre a fila e o tempo estimado de espera de postos do SAC Bahia.

4. Objetivos Específicos

- Criar uma API RESTful utilizando Node.js e Express.
- Permitir a consulta de postos e de suas respectivas filas.
- Persistir os dados dos postos e filas em um banco SQLite.
- Criar um cliente Python utilizando a biblioteca requests.
- Realizar a comunicação entre Python e Node.js por HTTP, utilizando JSON.
- Documentar e testar as rotas da API com OpenAPI/Swagger.
- Tratar respostas básicas de sucesso e erro, como posto inexistente.

5. Escopo do Projeto

5.1 Dentro do escopo

- Cadastro inicial de alguns postos fictícios/representativos do SAC para demonstração.
- Consulta da lista de postos disponíveis.
- Consulta de um posto específico pelo ID.
- Consulta da fila de um posto, incluindo quantidade de pessoas aguardando e tempo estimado.
- Atualização dos dados da fila para permitir a demonstração do sistema.
- Cliente Python em terminal para consultar o tempo de espera.
- Documentação das rotas com Swagger.

5.2 Fora do escopo

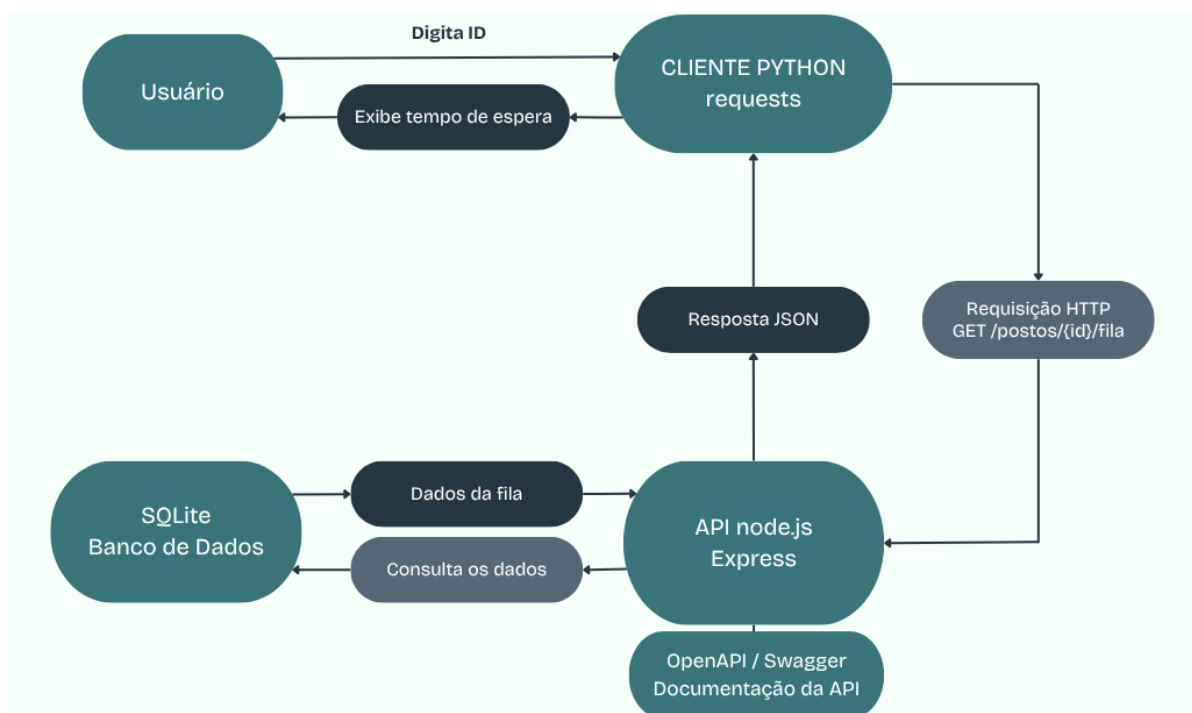
Para manter o TDE objetivo, não serão implementados autenticação de usuários, aplicativo móvel, interface web complexa, integração com sistemas reais do SAC, geolocalização, pagamentos, agendamentos reais ou infraestrutura em nuvem. Esses recursos poderão ser citados apenas como possíveis evoluções futuras.

6. Tecnologias

Tecnologia	Uso	Motivo
Node.js + Express	Servidor/API REST	Implementação simples de rotas HTTP.
Python + requests	Cliente consumidor	Realiza requisições HTTP de forma direta.
SQLite	Persistência	Banco leve, local e sem servidor adicional.
JSON	Formato de dados	Formato textual padrão para troca de dados REST.
OpenAPI/Swagger	Documentação	Permite visualizar e testar os endpoints da API.

7. System Design

A solução será dividida em componentes independentes. O cliente Python não acessará o banco de dados diretamente. Toda informação será obtida através da API REST, deixando explícita a comunicação entre processos e permitindo que cliente e servidor possam, conceitualmente, executar em máquinas diferentes.



7.1 Responsabilidade dos componentes

Componente	Responsabilidade
Cliente Python	Receber o ID digitado pelo usuário, chamar a API e apresentar o resultado no terminal.
API Node.js/Express	Receber requisições, validar parâmetros, consultar/atualizar dados e retornar JSON.
SQLite	Armazenar os dados dos postos e as informações atuais das filas.
Swagger	Descrever os endpoints, parâmetros, respostas e facilitar os testes.

8. Modelo de Dados Inicial

Para a primeira versão, uma estrutura simples é suficiente:

Campo	Tipo sugerido	Exemplo	Descrição
id	INTEGER	2	Identificador do posto.
nome	TEXT	SAC Feira de Santana	Nome do posto.
cidade	TEXT	Feira de Santana	Cidade do posto.
personas_fila	INTEGER	12	Quantidade aguardando.
tempo_estimado	INTEGER	25	Espera estimada em minutos.
atualizado_em	DATETIME	2026-09-14 09:30	Última atualização.

9. Endpoints Planejados

Método	Rota	Finalidade	Uso pelo Python
GET	/postos	Listar os postos disponíveis.	Opcional
GET	/postos/{id}	Consultar dados de um posto.	Opcional
GET	/postos/{id}/fila	Consultar fila e tempo estimado.	Principal
PUT	/postos/{id}/fila	Atualizar quantidade/tempo da fila.	Administração/teste

10. Exemplo de Comunicação

Requisição realizada pelo cliente:

GET /postos/2/fila

Resposta da API:

```
{  
  "posto": {
```

```

    "id": 2,
    "nome": "SAC Feira de Santana"
  },
  "fila": {
    "pessoas_aguardando": 12,
    "tempo_estimado_minutos": 25
  }
}

```

11. Requisitos Funcionais

- RF01 – O sistema deverá disponibilizar os postos cadastrados.
- RF02 – O sistema deverá permitir consultar um posto pelo seu ID.
- RF03 – O sistema deverá retornar o tempo estimado de espera da fila do posto.
- RF04 – O sistema deverá retornar a quantidade de pessoas aguardando.
- RF05 – O cliente Python deverá permitir que o usuário informe o ID do posto.
- RF06 – A API deverá permitir atualizar os dados da fila para fins de demonstração.

12. Requisitos Não Funcionais

- RNF01 – A comunicação entre cliente e servidor deverá utilizar HTTP/REST.
- RNF02 – As respostas da API deverão ser retornadas em JSON.
- RNF03 – A API deverá ser documentada utilizando OpenAPI/Swagger.
- RNF04 – O servidor será implementado em Node.js com Express.
- RNF05 – O cliente será implementado em Python utilizando requests.
- RNF06 – A persistência será realizada com SQLite.

13. Fluxo Principal

1. O usuário executa o cliente Python.
2. O programa apresenta os postos disponíveis ou solicita um ID.
3. O usuário informa o ID do posto desejado.
4. O Python envia uma requisição GET para a API Node.js.
5. A API valida o ID e consulta o SQLite.
6. A API retorna a resposta em JSON.
7. O Python interpreta a resposta e exibe o nome do posto, pessoas aguardando e tempo estimado.
8. Caso o ID não exista, o sistema informa o erro de forma adequada.

14. Planejamento de Implementação

Etapas	Atividade	Resultado esperado
1. Preparação	Criar estrutura do projeto e instalar dependências.	Ambientes Node.js e Python prontos.
2. Banco	Criar SQLite e inserir postos de	Dados persistidos.

	teste.	
3. API	Implementar e testar as rotas REST.	Endpoints funcionando em JSON.
4. Swagger	Documentar endpoints e respostas.	Documentação navegável/testável.
5. Cliente	Criar script Python com requests.	Consulta pelo ID funcionando.
6. Integração	Executar cliente e servidor separadamente.	Comunicação Node.js ↔ Python comprovada.
7. Testes	Testar IDs válidos, inválidos e atualização.	Demonstração estável.
8. Entrega	Revisar documentação e preparar apresentação.	Projeto pronto para avaliação.

15. Estrutura de Pastas Planejada

tde-sistemas-distribuidos/

```

├── servidor-node/
│   ├── src/
│   ├── database/
│   ├── app.js
│   └── package.json
├── cliente-python/
│   ├── cliente.py
│   └── requirements.txt
├── docs/
│   └── openapi.yaml
└── README.md

```

16. Critérios de Sucesso

O projeto será considerado funcional quando o servidor Node.js estiver executando independentemente, o cliente Python conseguir consultar um posto por meio de uma requisição HTTP, a API retornar dados válidos em JSON, as informações forem persistidas no SQLite e os endpoints estiverem descritos no Swagger/OpenAPI.

17. Possíveis Evoluções Futuras

Caso haja tempo após a implementação dos requisitos principais, poderão ser acrescentados recursos como histórico do tempo de espera, cálculo automático de estimativa, classificação dos postos por menor espera ou uma interface gráfica. Essas funcionalidades não fazem parte do escopo inicial e não são necessárias para a primeira entrega.

18. Resumo para Apresentação da Ideia

A equipe escolheu a Opção 2 – REST (OpenAPI/Swagger), utilizando o cenário de Fila de Espera do SAC Bahia. O sistema será composto por uma API em Node.js/Express e um cliente em Python. O usuário informará o ID de um posto no terminal Python, que consultará a API pela rede. A API buscará no SQLite a quantidade de pessoas na fila e o tempo estimado de espera e devolverá os dados em JSON. O Swagger será utilizado para documentar e testar as rotas. O escopo foi mantido objetivo para priorizar a integração entre serviços, que é o foco do TDE.